Multiprocessing in Python

CS3361 – Project #3 / Final Project

Dr. Eric Rees
Spring 2026

©2026 Eric Rees. All rights reserved.

Table of Contents

Introduction 3
Purpose of the Project 3
Educational Outcomes 3
Problem Statement 4
Guidance and Rules 4
Project Structure and Submission Requirements 4
Assignment 5 (Phase 1 – Serial Computation) 4
Final Project (Phase 2 – Concurrent Computation) 4
Requirements 5
Phase 1 – Serial Computation 5
Stage 1.1 – Data Retrieval 5
Stage 1.2 – Reading and Writing Matrix Files 5
Stage 1.3 – Matrix Processing 5
Phase 2 – Concurrent Computation 8
Stage 2.1 – Concurrency Using Multiprocessing 8
Command Line Arguments and Examples 8
Command Line Arguments 8
Example executions 9
File Specifications 9
Input File Specification 9
Output File Specification 10
Example Input/Output File 10
Additional Rules and Hints 10
Numeric Sequences 10
Powers of Two 10
Prime Numbers 10
Fibonacci Sequence 10
Program Output 11
Advice 11
Python Information 11
Artificial Intelligence (AI) Usage Policy 12
What to submit to RaiderCanvas 12

Assignment #5 Submission 12
Final Project Submission 12

Introduction

The final project in CS3361 introduces students to the complexities of parallel programming through Python's multiprocessing module. This project aims to provide students with their first experience in developing concurrent/parallel applications, setting a foundational understanding that will be further explored in-depth in CS 4352 (Operating Systems) and even more in depth in CS 4379 (Parallel Programming).

Purpose of the Project

The main objective of this project is to familiarize students with parallelism concepts and challenges, especially when working with a language that isn't naturally suited for intensive concurrent processing. While the first two projects utilized C/C++—languages well-suited for compiler design—this project introduces Python to solve a computationally demanding task. This contrast demonstrates how language selection impacts efficiency and performance, emphasizing the importance of choosing the right tool for the job. By implementing a modified cellular life simulator in Python, students will learn firsthand the limitations of Python in handling resource-intensive, concurrent applications, which will deepen their understanding of language selection in software development.

Educational Outcomes

Upon completion, students will:

- **Gain Practical Experience in Parallel Programming:** Understand how multiprocessing can enhance performance in computationally intensive applications, laying a foundation for advanced concurrency topics.

- **Analyze and Address Language Suitability:** Learn the trade-offs of using Python for tasks that demand high computational power and parallelism, expanding their critical skills in language evaluation.
    - By using Python for a task that demands high computational power, you'll learn about the language's strengths and limitations in handling multithreaded applications, broadening your perspective on language suitability for different types of problems.

- **Strengthen Problem-Solving Abilities:** Approach challenges of concurrency control, such as resource allocation, synchronization, and task distribution, in a language with inherent performance limitations.

- **Prepare for Operating Systems Coursework:** Develop familiarity with core concurrency concepts, easing their transition into more complex and low-level explorations in operating systems.

This project not only builds on skills acquired in the previous assignments but also serves as a key steppingstone into the world of parallelism and concurrency, fostering the analytical mindset needed to choose optimal programming languages and paradigms for future projects.

Problem Statement

You are tasked with creating a Python program capable of executing the first 100 iterations of a modified cellular life simulator. This simulator will receive the path to the input file as an argument containing the starting cellular matrix. The program must then simulate the next 100 iterations based on the algorithm discussed on the next few pages. The simulation is guided by a handful of simplistic rules that will result in a seemingly complex simulation of cellular organisms.

Guidance and Rules

The following two sections outline the guiding principles for your simulation application and suggest a sequential framework for tackling the problem. Your approach should be divided into two distinct phases: the serial computation phase and the concurrent computation phase. Each phase is broken into one or more stages and after completing each stage, it is highly recommended to rigorously test your program before proceeding to the subsequent one.

Project Structure and Submission Requirements

This project is divided into two parts, which will be submitted separately.

Assignment 5 (Phase 1 – Serial Computation)

Students are required to complete and submit Phase 1 of the project as Assignment 5. This phase focuses on implementing the full simulation using a single process (no multiprocessing).

Your submission for Assignment 5 must:

- Correctly implement all rules described in Phase 1 – Serial Computation
- Produce the correct final matrix after 100 iterations
- Not use any multithreading or the multiprocessing module

Final Project (Phase 2 – Concurrent Computation)

The Final Project builds directly on your Assignment 5 submission. In this phase, you will modify your existing solution to utilize multiprocessing.

Your submission for the Final Project must:

- Retain all functionality from Phase 1 – Serial Computation
- Incorporate multiprocessing as described in Phase 2 – Concurrent Computation
- Use the -p argument to control the number of processes

Your Final Project submission should reflect an extension of your Assignment 5 implementation.

Requirements

Phase 1 – Serial Computation

Stage 1.1 – Data Retrieval

Your solution should begin by parsing and verifying the command line arguments as they are described in the *Command Line Arguments* section on page 8.

Stage 1.2 – Reading and Writing Matrix Files

Stage 1.2.1 - Reading the Matrix File

Once your solution has verified the command line arguments, it should read in the matrix written into the input file given by the command line argument '-i'. This file will follow the specifications described in the *Input File Specification* section on page *9*. This matrix will represent the starting matrix (Time Step 0) from which matrix processing will occur as discussed in *Stage 1.3 – Matrix Processing*.

Stage 1.2.2 – Writing the Matrix File

Upon completion of all 100 generations from *Stage 1.3 – Matrix Processing*, your solution must write the generated matrix (Time Step 100) to an output file. The name and location of this output file is determined by the command line argument '-o'. This output file will follow the specification as described in the *Output File Specification* section on page 10.

Stage 1.3 – Matrix Processing

Your solution will accept the starting matrix (**Time Step 0**) from a file given in the '-i' command line argument and then simulate steps 1 through 100. The final matrix (**Time Step 100**) will then be written to an output file whose name and path is dictated by a separate command line argument. These files must contain a copy of your matrix with each row of the matrix printed on separate lines and no characters in between the columns.

Using the matrix retrieved from the input file given, your solution must then process the next 100 steps of a simulation that uses the following symbols and rules to dictate what occurs during each time step.

Stage 1.3.1 – Symbols

Your solution should be able to read and handle the following symbols, with each symbol representing a specific integer value.

- Cells that contain a capital letter 'O' are considered "Healthy O Cells" during this iteration.
- Cells that contain a capital letter 'X' are considered "Healthy X Cells" during this iteration.
- Cells that contain a lowercase letter 'o' are considered "Weakened O Cells" during this iteration.
- Cells that contain a lowercase letter 'x' are considered "Weakened X Cells" during this iteration.
- Cells that contain a period ('.') are considered "dead" during this iteration.

Stage 1.3.2 – Iterative Rules

Your solution must read from the matrix representing the current iteration and write all results to a separate matrix representing the next iteration. This ensures that each cell is evaluated independently using only the values from the current generation. The matrix must not be modified in place.

For each of the 100 iterations, apply the following rules to every cell in the matrix in the order listed below:

1. For each cell of the matrix, evaluate all neighboring cells within a radius of 2 (i.e., all cells in the surrounding 5×5 region centered on the current cell, excluding the current cell itself). These neighbors are divided into two groups:
    a. The inner ring, consisting of the 8 immediately adjacent cells.
    b. The outer ring, consisting of the remaining cells (up to 16) within the 5×5 region.

2. Each neighboring cell contributes a value based on its state:
    a. Neighboring cells that are "Healthy O Cells" are equal to three (+3).
    b. Neighboring cells that are "Weakened O Cells" are equal to one (+1).
    c. Neighboring cells that are "dead" are equal to zero (0).
    d. Neighboring cells that are "Weakened X Cells" are equal to negative one (-1).
    e. Neighboring cells that are "Healthy X Cells" are equal to negative three (-3).

3. Using these values, compute the following three scores:
    a. The inner score which is defined as the sum of all values from cells in the inner ring
    b. The outer score which is defined as the sum of all values from cells in the outer ring
    c. The influence score which is defined as: *influence score = (2 × inner score) + outer score*

4. If the current cell is a healthy 'O' cell
    a. If the "influence score" is a power of two, the cell dies.
    b. Otherwise, if the "outer score" is less than 2, the cell becomes a weakened 'o' cell.
    c. Otherwise, the cell remains healthy.

5. If the current cell is a weakened 'o' cell
    a. If the "inner score" is a member of the Fibonacci sequence, the cell becomes a healthy 'O' cell.
    b. Otherwise, if the "influence score" is less than or equal to 0, the cell dies.
    c. Otherwise, the cell remains weakened.

6. If the current cell is a dead cell
    a. If the "influence score" is a prime number, the cell becomes a weakened 'o' cell.
    b. Otherwise, if the "influence score" is negative and the absolute value of the "influence score" is a prime number, the cell becomes a weakened 'x' cell.
    c. Otherwise, the cell remains dead.

7. If the current cell is a weakened ‘x’ cell
    a. If the “inner score” is negative and the absolute value of the “inner score” is a member of the Fibonacci sequence, the cell becomes a healthy ‘X’ cell.
    b. Otherwise, if the “influence score” is greater than or equal to 0, the cell dies.
    c. Otherwise, the cell remains weakened.

8. If the current cell is a healthy ‘X’ cell
    a. If the “influence score” is negative and the absolute value of the “influence score” is a power of two, the cell dies.
    b. Otherwise, if the “outer score” is greater than -2, the cell becomes a weakened ‘x’ cell.
    c. Otherwise, the cell remains healthy.

For this program, a neighbor is defined as any cell within a radius of 2 from the current cell (C), excluding the cell itself. This forms a 5×5 region centered on the current cell. Neighbors are divided into two groups:

- **N1 (Inner Ring):** The 8 cells immediately adjacent to the current cell.
- **N2 (Outer Ring):** The remaining cells within the 5×5 region that are not part of N1.

Cells near the edges of the matrix will have fewer neighbors. As a result, inner and outer scores may be computed from fewer than 8 and 16 cells respectively for edge and corner positions.

The examples below illustrate the N1 and N2 neighbors for a given cell (C). Example #1 shows a cell located in the middle of the matrix, while Example #2 shows a cell near the edge of the matrix.

Neighbor Example #1

	0	1	2	3	4	5
0	N2	N2	N2	N2	N2	
1	N2	N1	N1	N1	N2	
2	N2	N1	C	N1	N2	
3	N2	N1	N1	N1	N2	
4	N2	N2	N2	N2	N2	
5						

Neighbor Example #2

	0	1	2	3	4	5
0						
1						
2						
3				N2	N2	N2
4				N2	N1	N1
5				N2	N1	C

Phase 2 – Concurrent Computation

Stage 2.1 – Concurrency Using Multiprocessing

Once Phase 1 has been completed, the next task is to re-write *Stage 1.3 – Matrix Processing* to now make use of currency via the *multiprocessing* module. As such, your solution is required to effectively utilize the Python *multiprocessing* module to enhance its performance. It should be capable of initializing a number of processes that correspond to the number specified by the user through the `-p` option. Typically, when not using the *multiprocessing* module, your program executes serially—that is, it operates using a single thread or process. However, for the purpose of this project, you must adapt your program to run in parallel using multiple processes. To comply with this requirement, ensure that your solution:

- Parses the `-p` option to determine the number of processes the user intends to create.
- Implements the `multiprocessing` module to spawn the exact number of processes requested by the user and utilizes all of them.
- Orchestrates these processes to work concurrently on the task at hand, which should demonstrate a clear understanding and application of parallel processing concepts.

Proper implementation of the *multiprocessing* module is crucial. If your solution does not conform to these multiprocessing standards, it will be deemed incorrect. Make sure to test and verify that each process is performing its intended task and that all processes are running in parallel as expected.

Command Line Arguments and Examples

Command Line Arguments

Develop a program capable of accepting the following command line arguments:

- -i <path_to_input_file>
  - Purpose: This option retrieves the file path to the starting cellular matrix.
  - Input Type: String
  - Validation: Entire file path must exist, otherwise error.
  - Required: Yes
- -o <path_to_output_file>
  - Purpose: This option retrieves the file path for the final output file.
  - Input Type: String
  - Validation: The directories in the file path must exist, otherwise error.
  - Required: Yes
- -p <int>
  - Purpose: This option retrieves the number of processes to spawn.
  - Input Type: Unsigned Integer
  - Validation: Must be a positive integer > 0, otherwise error.
  - Required: No
  - Default Value: 1

Example executions

- *python3 Eric_Rees_R123456_final_project.py -i inputFile.txt -o timeStep100.txt -p 36*
  - Sets input file to "inputFile.txt"
  - Sets output file to "timeStep100.txt"
  - Sets process count to 36
- *python3 Eric_Rees_R123456_final_project.py -o myOutput.txt -i myInput.txt*
  - Sets input file to "myInput.txt"
  - Sets output file to "myOutput.txt"
  - Sets process count to 1 (default when not specified)

File Specifications

Input File Specification

The input file must be an ASCII text file containing a representation of a matrix, with each cell within the matrix represented by specific symbols:

- Healthy O Cells are represented by a capital letter 'O'.
- Healthy X Cells are represented by a capital letter 'X'.
- Weakened O Cells are represented by a lowercase letter 'o'.
- Weakened X Cells are represented by a lowercase letter 'x'.
- Dead Cells (Dead) represented by a period ('.').

The matrix must abide by the following rules:

- Delimiters
  - **Between cells:** No spaces, commas, or any other delimiters should be used between the symbols representing cells. Each symbol directly follows the previous symbol within a row.
  - **Between rows:** Each row of the matrix is delimited by an End of Line (EOL) character. This applies to all rows except potentially the final row.
    - This specification does not mandate a specific EOL character due to differences between operating systems (e.g., \n for Unix/Linux, \r\n for Windows). However, files should be consistent in their use of EOL characters.
    - The final row in the matrix may optionally terminate with an End of Line character. This is permissible but is <u>not</u> required.
- Matrix Dimensions
  - The matrix can have any number of rows and columns, but each row must have the same number of symbols (cells) to form a proper rectangular matrix.
- Invalid Symbols
  - Any symbols other than those listed above ('O', 'o', 'X', 'x', '.') within the matrix are considered invalid and may lead to errors in processing the file.

Output File Specification

The output file matches the same specifications as the input file – see Input File Specification above.

Example Input/Output File

```
..Oo..
O.....
...x..
....O.
....X.
.O..x.
```

Additional Rules and Hints

Numeric Sequences

This project requires checking if numbers are either powers of two, prime, or members of the Fibonacci sequence. As such, these numeric sets are defined below to alleviate any confusion:

Powers of Two

A "power of two" is a natural number of the form 2^n, where n is a non-negative integer. These are important in computer science because binary systems are based on powers of two.

Prime Numbers

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. For example, 2 is a prime number because it only evenly divisible by itself (2) and 1. In fact, two is the only even prime number because all other even numbers are divisible by 2 (e.g. 4 is divisible by 4, 2, and 1 – which is more than just 4 and 1).

Fibonacci Sequence

The Fibonacci sequence is a sequence of numbers where each number is the sum of the two preceding ones. It starts like this:

$$F_0 = 0, \qquad F_1 = 1, \qquad F_n = F_{n-1} + F_{n-2}$$

So, the sequence goes:

$$0, 1, 1, 2, 3, \ldots$$

Each number is the sum of the previous two:

$$2 = 1 + 2, \qquad 3 = 2 + 1, \qquad \ldots$$

Program Output

Your program may print as much information as you wish to the screen during execution. The first line of output printed to the screen during execution **must** be "Project :: R#" where R# is your specific TTU R#. That is the only **required** item it must print and is the only line the grading script cares about. Your program correctness grade will be determined by the output found in the file specified by the -o argument. Your output file should **only** contain the matrix generated at time step 100 with no other extraneous information.

Advice

Leave room for testing! This application is meant to give you a taste for how intense large-scale computations can be. As such, testing a poorly optimized solution could take up to 48 hours to execute and may not have completed successfully in that time frame. Make sure you leave at least **7 days** for testing. Waiting to test your program a few days before it is due is a guaranteed way of not completing this project on time.

Python Information

1) Warning about compatibility:
    a. Your solution must be written to be compatible with Python version 3.x with only the base Python3 modules.
        i. Specifically, the grading script will be running a fresh install of Python 3.13.2.
        ii. Keep in mind, this means you cannot use Numpy, Scipy or other possibly useful matrix modules.
    b. Python version 2.x was ubiquitous and heavily documented online for almost two decades.
    c. Python version 2.x code is often not compatible with Python version 3.x.
2) Python 3.x Tutorial / Refresher Links:
    a. https://www.w3schools.com/python/default.asp
    b. https://www.tutorialspoint.com/python3/index.htm
3) Python 3.x Tutorial for handling command line arguments:
    a. https://docs.python.org/3/howto/argparse.html
    b. https://pymotw.com/3/argparse/
4) Additional information and tutorials for multiprocessing:
    a. The link below covers the multiprocessing module in considerable detail:
        i. Multiprocessing - http://zetcode.com/python/multiprocessing/

Artificial Intelligence (AI) Usage Policy

The use of artificial intelligence tools is **not permitted for this project**.

This includes, but is not limited to:

- ChatGPT
- GitHub Copilot
- Claude
- Gemini
- Any other AI system capable of generating, modifying, or explaining code

Your implementation must be written entirely by you without the assistance of AI tools.

Students remain responsible for understanding every part of their submitted code. During grading, students may be asked to briefly explain how portions of their program work.

Submissions that appear inconsistent with a student's demonstrated understanding may be subject to review under the Texas Tech University Academic Integrity Policy and any applicable course policies.

What to submit to RaiderCanvas

Assignment #5 Submission

A zip archive (.zip) which contains:

- The primary python file for your project named <FirstName>_<LastName>_<R#>_assignment_5.py
  - Example: Eric_Rees_R123456_ assignment_5.py
  - Keep in mind the grading program will require that your python file be named appropriately.
- Any additional files necessary for your program to execute should be included.

Final Project Submission

A zip archive (.zip) which contains:

- The primary python file for your project named <FirstName>_<LastName>_<R#>_final_project.py
  - Example: Eric_Rees_R123456_final_project.py
  - Keep in mind the grading program will require that your python file be named appropriately.
- Any additional files necessary for your program to execute should be included.